

OAT 1 – Introdução a Diagrama de Classes

Autores:

<Ian Dias Duque>

<Olavo Barros Meira Filho>

Agenda

O objetivo dessa apresentação é
....

1. Item 1:
Contexto do projeto

2. Item 2:
Objetivo e escopo da API

3. Item 3:
Principais funcionalidades

4. Item 4:
Modelagem conceitual
(Diagrama de Classes)

5. Item 5:
End points

OBJETIVO DO SISTEMA

Itens para discussão:

- Desenvolver uma API REST em Java
- Gerenciar usuários, eventos e ingressos
- Aplicar conceitos de orientação a objetos
- Implementar regras de negócio definidas nas User Stories
- Servir como base para futuras evoluções da plataforma



FUNCIONALIDADES PRINCIPAIS



- **Cadastro, ativação e desativação de usuários**
- **Cadastro e gerenciamento de eventos**
- **Feed de eventos ativos**
- **Compra e cancelamento de ingressos**
- **Listagem de ingressos por usuário**

REGRAS DE NEGÓCIO



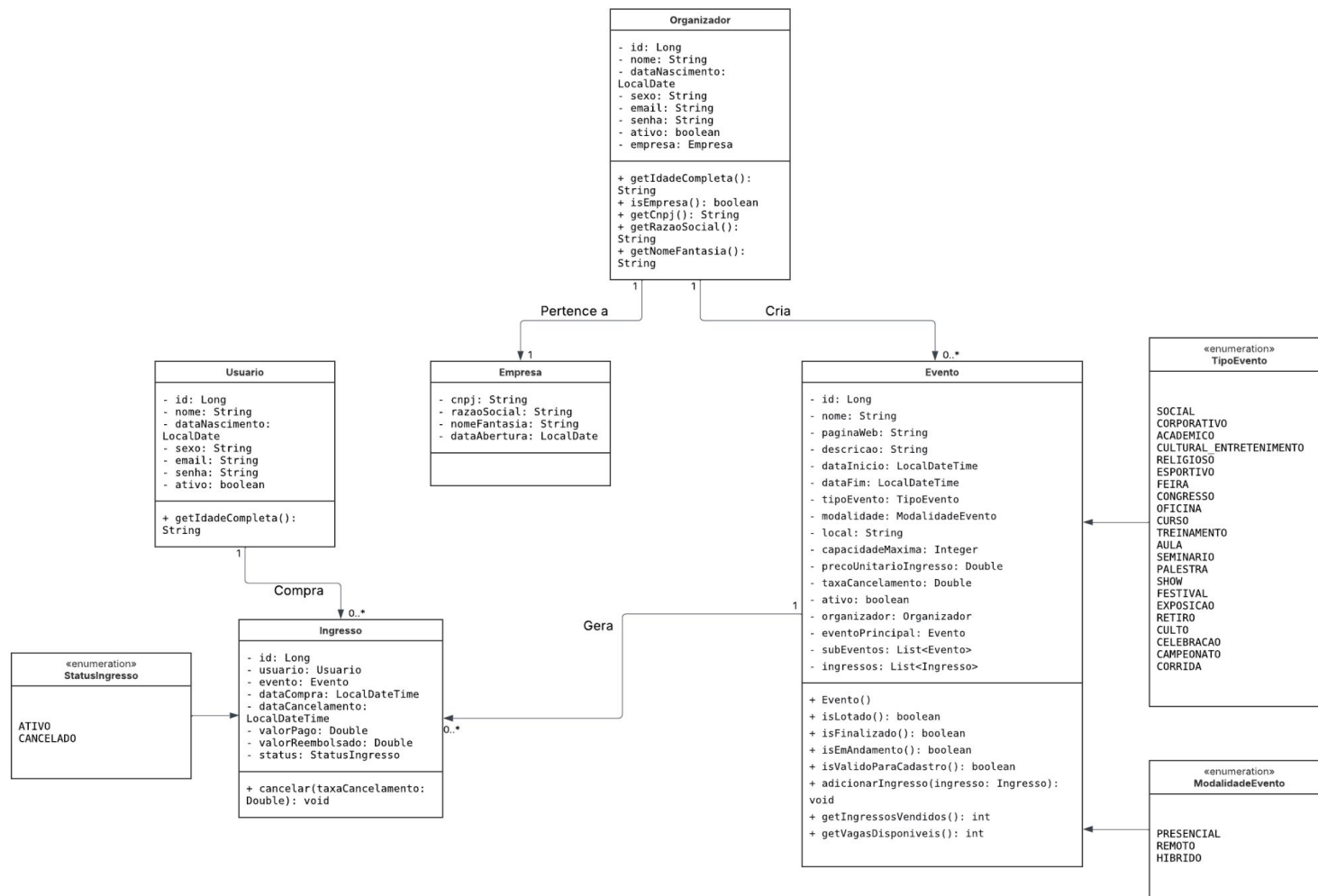
Pontos de interesse:

- Não é permitido usuários com e-mail duplicado
- Eventos não podem iniciar em datas passadas
- Eventos devem ter duração mínima de 30 minutos
- Eventos com ingressos vendidos exigem reembolso ao serem desativados
- Organizadores só podem desativar conta se não tiverem eventos ativos

MODELAGEM CONCEITUAL



- A modelagem do sistema foi realizada por meio de um Diagrama de Classes, representando as principais entidades do domínio, como Usuário, Organizador, Evento e Ingresso, além de seus atributos, métodos e relacionamentos.
- Esse diagrama serve como base para a implementação da API e garante maior clareza na estrutura do sistema.



VERBOS HTTP



VERBO	USO NA API
POST	Criar novos recursos
GET	Consultar informações
PUT	Atualizar dados
PATCH	Alterar status
POST	Executar ações (ex: comprar ingresso)

CONSTRUÇÃO DE ROTAS (USUÁRIO)



VERBO	ROTA
POST	/usuarios
GET	/usuarios/{usuariold}
PUT	/usuarios/{usuariold}
PATCH	/usuarios/{usuariold}/{status}

CONSTRUÇÃO DE ROTAS (ORGANIZADOR)



VERBO	ROTA
POST	/organizadores
GET	/organizadores/{organizadorId}
PUT	/organizadores/{organizadorId}
PATCH	/organizadores/{organizadorId}/{status}

CONSTRUÇÃO DE ROTAS (EVENTO)



VERBO	ROTA
POST	/organizadores/{organizadorId}/eventos
GET	/organizadores/{organizadorId}/eventos
PUT	/organizadores/{organizadorId}/eventos/{eventId}
PATCH	/organizadores/{organizadorId}/eventos/{eventId} /{status}

CONSTRUÇÃO DE ROTAS (INGRESSO)



VERBO	ROTA
POST (comprar ingresso)	/organizadores/{organizadorId}/eventos/{eventoId}/ingressos
POST (cancelar ingresso)	/usuarios/{usuarioId}/ingressos/{ingressoId}
GET	/usuarios/{usuarioId}/ingressos

EXEMPLO DE CÓDIGO



```
@PatchMapping(path =("/{usuarioId}/{status}") no usages
public ResponseEntity<String> alterarStatus(
    @PathVariable(parameter = "usuarioId") long usuarioId,
    @PathVariable(parameter = "status") String status) {

    Optional<Usuario> usuarioOpt = repositorio.buscarUsuarioPorId(usuarioId);

    if (!usuarioOpt.isPresent()) {
        return ResponseUtils.badRequest("Usuário não encontrado com ID: " + usuarioId);
    }

    Usuario usuario = usuarioOpt.get();

    if ("ativar".equalsIgnoreCase(status)) {
        usuario.setAtivo(true);
        repositorio.salvarUsuario(usuario);
        return ResponseUtils.ok("Usuário reativado com sucesso!");
    } else if ("desativar".equalsIgnoreCase(status)) {
        usuario.setAtivo(false);
        repositorio.salvarUsuario(usuario);
        return ResponseUtils.ok("Usuário desativado com sucesso!");
    } else {
        return ResponseUtils.badRequest("Status inválido. Use 'ativar' ou 'desativar'");
    }
}
```

REQUISIÇÃO E RESPONSE POST (USUÁRIOS)



The screenshot displays a REST client interface with a sidebar on the left showing a collection of endpoints under 'Dendê Softhouse' > 'usuários'. The main panel shows a POST request to 'http://localhost:8080/usuarios' with a JSON body. The response is a 200 OK status with a message in Portuguese.

Request:

- Method: POST
- URL: http://localhost:8080/usuarios
- Body (raw):

```
1 {  
2   "nome": "Lucas Almeida Silva",  
3   "dataNascimento": "1994-11-11",  
4   "sexo": "M",  
5   "email": "lucas@mail.com",  
6   "senha": "senha123"  
7 }
```

Response:

- Status: 200 OK
- Time: 556 ms
- Size: 130 B
- Body (raw):

```
1 "Usuario lucas@mail.com registrado com sucesso! ID: 1"
```

REQUISIÇÃO E RESPONSE POST (ORGANIZADORES)



The screenshot displays a REST client interface with a sidebar on the left and a main workspace. The sidebar shows a collection named 'Dendê Softhouse' with sub-items: 'usuários', 'ingressos', 'eventos', and 'organizadores'. The 'organizadores' collection is expanded, showing a list of actions: 'POST Cadastro de Usuário Organizador', 'PUT Alterar Usuário Organizador', 'PATCH Desativar ou Reativar Organizador', and 'GET Visualizar Perfil do Organizador'. The main workspace shows a POST request to 'http://localhost:8080/organizadores'. The request body is a JSON object with the following fields: 'nome', 'dataNascimento', 'sexo', 'email', 'cnpj', 'razaoSocial', 'nomeFantasia', and 'senha'. The response is a 200 OK status with a message: 'Organizador lucas@gmail.com registrado com sucesso! ID: 1'.

Request:

```
POST http://localhost:8080/organizadores
```

Body:

```
{  2    "nome": "Lucas Almeida Silva",
  3    "dataNascimento": "1994-11-11",
  4    "sexo": "M",
  5    "email": "lucas@gmail.com",
  6    "cnpj": "",
  7    "razaoSocial": "",
  8    "nomeFantasia": "",
  9    "senha": "senha123"
10 }
```

Response:

```
1  "Organizador lucas@gmail.com registrado com sucesso! ID: 1"
```

REQUISIÇÃO E RESPONSE POST (EVENTOS)



+ Search collections

▼ Dendê Softhouse

> usuarios

> ingressos

▼ eventos

POST Cadastrar Evento

PUT Alterar Evento

PATCH Desativar ou Ativar Evento

GET Listar Eventos do Organizador

GET Feed de Eventos

> organizadores

> My Collection

HTTP Dendê Softhouse / eventos / Cadastrar Evento

Save

Share

POST

http://localhost:8080/organizadores/{{usuarioid}}/eventos

Send

Docs

Params

Authorization

Headers (8)

Body

Scripts

Tests

Settings

Cookies

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

Schema

Beautify

```
1 {
2   "nome": "Meu Evento de Teste",
3   "paginaWebEvento": "https://www.meueventoteste.com",
4   "descricao": "Esse é um evento de teste para a minha plataforma. Vamos ter várias atrações",
5   "tipoEvento": "CULTURAL_ENTRETENIMENTO",
6   "modalidade": "PRESENCIAL",
7   "capacidadeMaxima": 200,
8   "dataInicio": "2026-03-30T18:50:00",
9   "dataFim": "2026-03-30T19:30:00",
10  "precoUnitarioIngresso": 125.2
11 }
```

Body

Cookies

Headers (2)

Test Results

200 OK

8 ms

134 B

Save Response

Raw

Preview

Visualize

```
1 "Evento Meu Evento de Teste cadastrado com sucesso! ID: 1"
```


REQUISIÇÃO E RESPONSE POST (INGRESSOS)



The screenshot displays a REST client interface with a sidebar on the left and a main workspace on the right.

Sidebar:

- Search collections
- Dendê Softhouse
 - usuários
 - ingressos
 - POST Comprar Ingresso**
 - POST Cancelar Ingresso
 - GET Listar Ingressos do Usuário
 - eventos
 - organizadores
- My Collection

Main Workspace:

HTTP Dendê Softhouse / ingressos / **Comprar Ingresso** [Save] [Share]

POST `http://localhost:8080/organizadores/ {{organizadorId}} /eventos/ {{eventoid}} /ingressos` [Send]

Tabs: Docs Params Authorization Headers (8) **Body** Scripts Tests Settings Cookies

Body format: ☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** [Schema] [Beautify]

```
1 {
2   "usuario": "lucas@mail.com",
3   "totalPago": 125.20
4 }
```

Response: 200 OK • 20 ms • 114 B • [Save Response]

Body: Raw [Preview] [Visualize]

```
1 "Ingresso comprado com sucesso! ID: 1"
```

REFERÊNCIAS



Documentação da OAT 1 – Introdução a Diagrama de Classes

Repositório do projeto Dendê Eventos API

Material didático da disciplina

Modelo Conceitual fornecido pela Dendê Softhouse